

BÁO CÁO BÀI TẬP

Môn học: Kỹ thuật phân tích mã độc

Bài tập nhóm 02

GVHD: [Nguyễn Công Danh](#)

NHÓM: 11

1. THÔNG TIN CHUNG:

(Liệt kê tất cả các thành viên trong nhóm)

Lớp: NT137.Q11.ANTT

STT	Họ và tên	MSSV	Email
1	Đồng Hữu Nguyên Khoa	23520734	23520734@gm.uit.edu.vn
2	Trịnh Nhật Duy	23520394	23520394@gm.uit.edu.vn
3	Nguyễn Phan Quốc Đạt	22521203	22521203@gm.uit.edu.vn
4	Nguyễn Quang Lộc	23520860	23520860@gm.uit.edu.vn

2. NỘI DUNG THỰC HIỆN:¹

STT	Công việc	Kết quả tự đánh giá
1	Bài tập 1	100%

Phần bên dưới của báo cáo này là tài liệu báo cáo chi tiết của nhóm thực hiện.

¹ Ghi nội dung công việc, các kịch bản trong bài Thực hành

Danh mục

1. Kịch bản 01	4
2. Luồng hoạt động của một công cụ đóng gói (packer) và giải nén (unpacker/stub) cho tệp PE (Portable Executable):.....	4
a) Cấu Trúc Cơ Bản	4
b) Luồng Hoạt Động (Workflow)	5
3. Tiến hành Implement công cụ đơn giản:.....	8
a) Import Table.....	8
b) Relocation	8
c) Triển khai các công việc còn lại	10
4. Test với VirusTotal	14

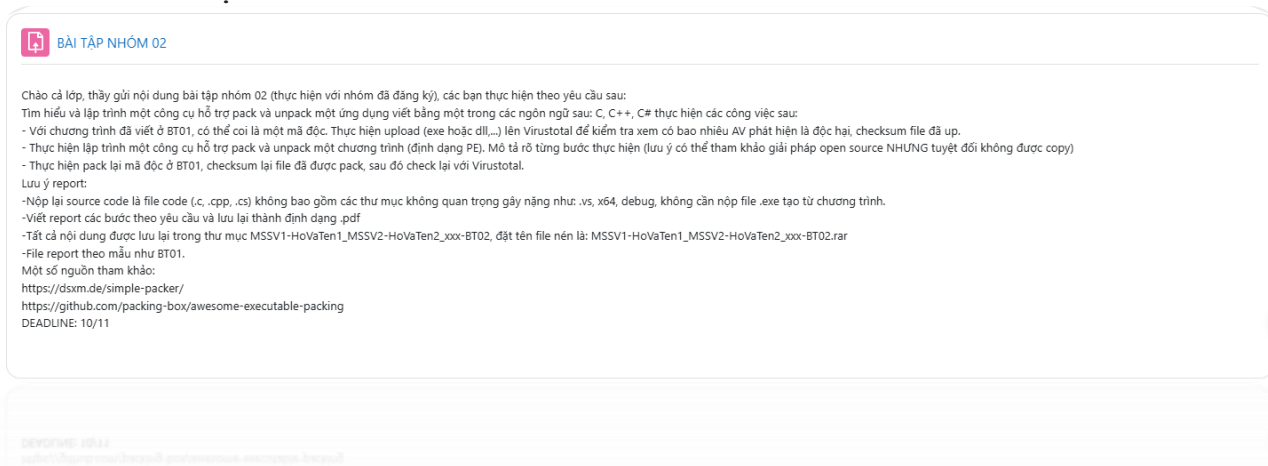
Danh mục hình ảnh

Hình 1. Yêu cầu bài tập.....	4
Hình 2. Workflow.....	5
Hình 3. Minh họa về IAT	5
Hình 4. Mô phỏng quá trình Unpack	8
Hình 5. Cấu trúc cơ bản của IMAGE_IMPORT_DESCRIPTOR.....	8
Hình 6. Function để giải quyết IAT	8
Hình 7. Mô tả đơn giản về Relocation	9
Hình 8. Function để giải quyết Relocation	9
Hình 9. Function giúp thực thi trong bộ nhớ ảo	11
Hình 10.Format Packed File	11
Hình 11. Function xử lý đóng gói file.....	12
Hình 12. Function xử lý việc unpack và giúp file execute	12
Hình 13. File packed được insert thêm.....	13
Hình 14. Marker được chèn vào phần cuối file	13
Hình 15. Separator được chèn vào trước phần được encrypt	13
Hình 16. Các DLL được gọi.....	13
Hình 17. Các winhttp.dll được gọi.....	14
Hình 18. Usage.....	14
Hình 19. Kết quả file vẫn có thể tự unpack và execute	15
Hình 20. Kiểm tra các Registry và vẫn đăng ký được	15
Hình 21. Checksum.....	15
Hình 22. Packed File	16
Hình 23. Original File	16

BÁO CÁO CHI TIẾT

1. Kịch bản 01

- Tài nguyên:
- Mô tả/mục tiêu:



Hình 1. Yêu cầu bài tập

- Các bước thực hiện/ Phương pháp thực hiện (Ảnh chụp màn hình, có giải thích)

Ở đây vì nội dung khá nhiều nên nhóm sẽ chia ra làm nhiều phần riêng như sau:

Đầu tiên, chúng ta sẽ tìm hiểu về cách thức cũng như luồng hoạt động(workflow) của một công cụ pack/unpack PEFile.

Tiếp theo đó, sau khi đã tìm hiểu về workflow của công cụ, chúng ta sẽ đi từng bước để viết nên một công cụ đơn giản.

2. Luồng hoạt động của một công cụ đóng gói (packer) và giải nén (unpacker/stub) cho tệp PE (Portable Executable):

Hãy hình dung quá trình đóng gói một tệp PE giống như việc chúng ta **đóng gói một căn nhà (File gốc) vào một hộp bí mật (File đã nén)** để vận chuyển và xây dựng lại nó ở nơi khác.

a) Cấu Trúc Cơ Bản

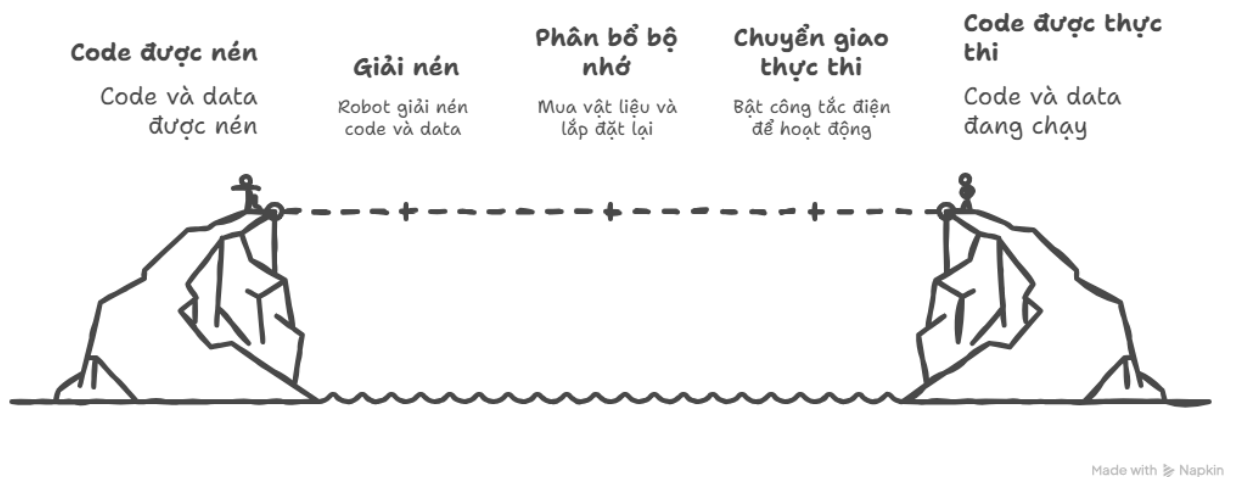
Hệ thống thường được chia làm hai tác vụ chính, bao gồm đóng gói(pack) và giải nén(unpack):

- Ở giai đoạn pack, các tác vụ sẽ được thực hiện hầu hết ở máy của chính người tạo ra file gốc. Nhiệm vụ của nó là xử lý file gốc (ngôi nhà) và tạo ra tệp đã đóng gói (hộp bí mật).

- Giai đoạn thứ hai là unpack, tại đây hầu hết sẽ được thực hiện ở máy victim. Đây là đoạn mã **nhỏ, độc lập** được chèn vào file đã đóng gói. Nó giống như một **cơ chế tự động** được kích hoạt khi hộp bí mật được mở.

b) Luồng Hoạt Động (Workflow)

Quá trình này giống như việc chuyển nhà: Packer nén toàn bộ đồ đạc (code/data) vào một container (packed section), kèm theo một "robot xây dựng" nhỏ (Stub) và một cuốn sổ tay hướng dẫn (metadata). Khi container đến nơi, robot tự giải nén đồ đạc, tự động mua vật liệu (LoadLibraryA/GetProcAddress), lắp đặt lại ngôi nhà trong lô đất trống (VirtualAlloc) và cuối cùng là bật công tắc điện (chuyển giao thực thi về OEP) để ngôi nhà hoạt động bình thường.



Hình 2. Workflow

- I. **Giai đoạn Đóng gói (Packing Stage):** Giai đoạn này còn được gọi là obfuscation, bao gồm việc đưa file thực thi gốc thành một stub executable (tệp thực thi chứa mã giải nén).
 1. Xóa bỏ hoặc làm hỏng Bảng Địa chỉ Import (IAT) và Bypass AV. IAT là một bảng chứa thông tin để chỉ dẫn Windows loader tải và giải quyết các thư viện DLL cùng các địa chỉ hàm API tương ứng

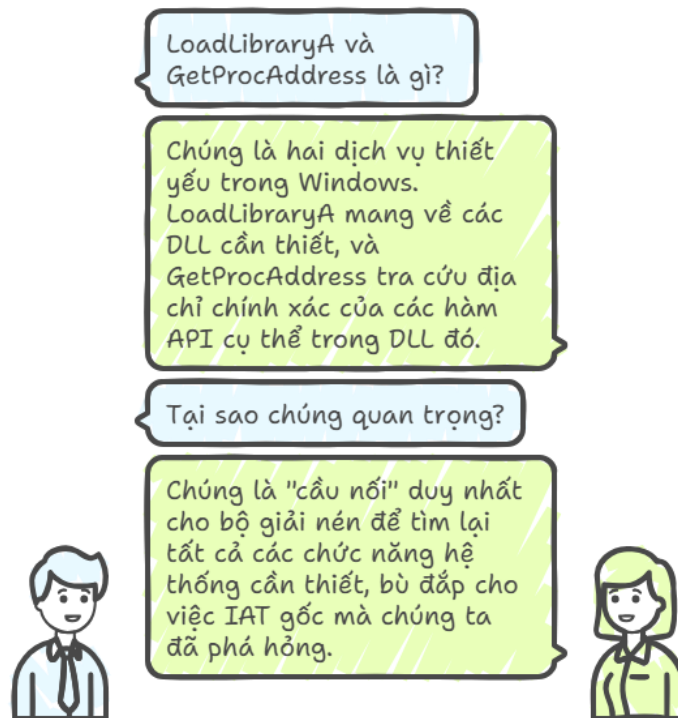
000000014000C780	__gox_personality_seh0	libstdc++-6
WINHTTP		
000000014000C790	WinHttpCloseHandle	WINHTTP
000000014000C798	WinHttpConnect	WINHTTP
000000014000C7A0	WinHttpOpen	WINHTTP
000000014000C7A8	WinHttpOpenRequest	WINHTTP
000000014000C7B0	WinHttpQueryDataAvailable	WINHTTP
000000014000C7B8	WinHttpReadData	WINHTTP
000000014000C7C0	WinHttpReceiveResponse	WINHTTP
000000014000C7C8	WinHttpSendRequest	WINHTTP
000000014000C7D0	WinHttpSetOption	WINHTTP

Hình 3. Minh họa về IAT

2. Khi IAT gốc bị xóa, packer cần đảm bảo rằng **stub** (mã giải nén) có thể truy cập các hàm cơ bản để bắt đầu quá trình giải nén và xây dựng lại. Để làm được việc này, chúng ta cần lưu thông tin Import vào Section mới (.packed) và tạo IAT mới chỉ bao gồm các địa chỉ của các hàm cơ bản như **LoadLibraryA** và **GetProcAddress** từ **kernel32.dll**.

Giải thích một chút về các hàm **LoadLibraryA** và **GetProcAddress**. Hiểu đơn giản như chúng ta đang xây dựng một tòa nhà (file thực thi) trong một thành phố mới (Windows). Thay vì phải ghi sẵn địa chỉ (hardcoded) của mọi dịch vụ (API) ngay từ đầu (vì địa chỉ đó có thể thay đổi trong các phiên bản thành phố khác nhau), chúng ta chỉ cần ghi nhớ địa chỉ của **hai dịch vụ thiết yếu**:

LoadLibraryA (dịch vụ Bưu điện, giúp mang về các cuốn Danh bạ/DLL cần thiết) và **GetProcAddress** (dịch vụ Thông tin, giúp tra cứu địa chỉ chính xác của mọi văn phòng/hàm API cụ thể trong cuốn danh bạ đó). Tóm lại, **LoadLibraryA** và **GetProcAddress** là bộ đôi API đóng vai trò là "cầu nối" duy nhất cho bộ giải nén để tìm lại tất cả các chức năng hệ thống cần thiết, bù đắp cho việc IAT gốc mà chúng ta đã phá hỏng ở bước trên.

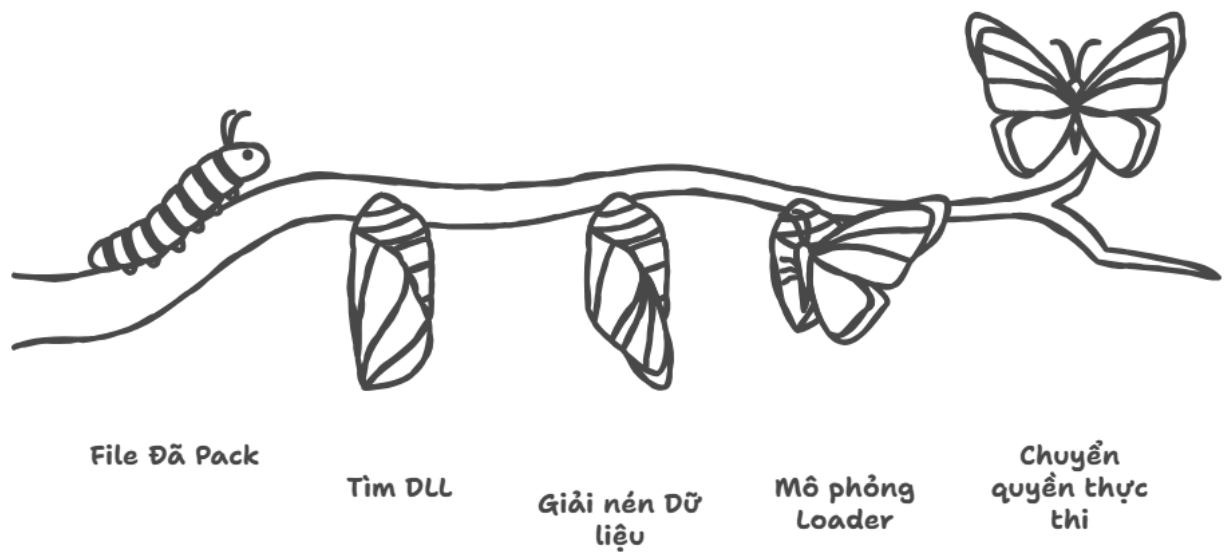


3. Sau khi lưu trữ thông tin Import và tạo IAT mới, chúng ta sẽ nén binary và đặt vào Section mới. Tất cả dữ liệu thô của các section gốc được **đọc, tập hợp thành một khối (buffer) duy nhất và được nén lại**. Tại đây, sẽ bao gồm các thông tin để cho giai đoạn **Unpack**.
- II. Giai đoạn Giải nén và Thực thi (Unpacking and Execution Stage):** Khi Packed.exe(kết quả của giai đoạn pack) được chạy, Windows Loader sẽ nạp

nó vào bộ nhớ và thực thi từ New Entry Point (tức là Unpacking Stub). Stub này phải thực hiện các bước sau:

- 1) Stub tìm kiếm DLL cơ bản và tự xây dựng lại code gốc: thông qua các địa chỉ được nạp trong Windows Loader, gọi các hàm cơ bản như **LoadLibraryA** và **GetProcAddress** để bổ sung lại các DLL cần thiết.
- 2) Giải nén dữ liệu:
 - Stub tìm section chứa dữ liệu đã nén.
 - Stub đọc kích thước gốc (file trước khi pack) và sử dụng thuật toán giải nén tương ứng để giải nén toàn bộ dữ liệu file PE gốc (bao gồm cả các sections và header) vào một **vùng bộ nhớ mới** (ImageBase) được cấp phát bằng VirtualAlloc.
- 3) Mô phỏng lại Windows Loader: quá trình này Stub sẽ thực hiện các công việc để khôi phục lại thực thi file gốc như tạo VirtualAlloc, ánh xạ Section,...
- 4) Cuối cùng là giải quyết relocation và chuyển quyền thực thi sang OEP.

*Giải thích về OEP: OEP chính là **khởi điểm** mà Windows Loader chuyển giao quyền kiểm soát để chương trình bắt đầu hoạt động. Có thể hiểu đơn giản nếu coi chương trình là một cuộc đua marathon, **OEP** là vạch xuất phát thực sự của vận động viên. Khi file bị nén, packer chuyển vạch xuất phát tạm thời sang vị trí của **đoạn mã giải nén (stub)**. Đoạn mã stub hoạt động như một "người khôi phục" và sau khi làm xong công việc của mình, nó sẽ đưa chương trình trở lại vạch xuất phát gốc (OEP) để chương trình có thể chạy bình thường.*



Made with Napkin

Hình 4. Mô phỏng quá trình Unpack

3. Tiến hành Implement công cụ đơn giản:

Theo như cơ sở lý thuyết mà chúng ta đã nói bên phần trên, phần khó và phức tạp nhất là giải quyết Import Table và tính toán sự chênh lệch về địa chỉ(relocation). Chúng ta sẽ tập trung vào hai phần này.

a) Import Table

b) Relocation

Khi **linker** tạo ra một file **EXE** hoặc **DLL**, nó phải giả định file sẽ được nạp vào một địa chỉ ưu tiên trong bộ nhớ, gọi là ImageBase (ví dụ: 0x400000 cho file EXE). Dựa trên ImageBase này, **linker** có thể tạo ra các địa chỉ tuyệt đối được "**hard-code**" ngay trong dữ liệu của chương trình.

Vậy thì có một vấn đề xảy ra, nếu địa chỉ ImageBase ưu tiên đó đã bị một file DLL khác chiếm mất khi chương trình execute. Windows Loader sẽ phải nạp file vào một địa chỉ khác. Lúc này, tất cả các địa chỉ tuyệt đối đã được hard-code trước đó sẽ bị sai, và chương trình gần như sẽ bị crash.



Hình 7. Mô tả đơn giản về Relocation

Vậy thì chúng ta có thể triển khai như sau:

```
void processRelocations(LPVOID pImageBase, IMAGE_NT_HEADERS* ntHeaders) {
    IMAGE_DATA_DIRECTORY& relocDir = ntHeaders->OptionalHeader.DataDirectory[IMAGE_DIRECTORY_ENTRY_BASERELOC];
    if (relocDir.VirtualAddress == 0) return;

    ULONG_PTR delta = (ULONG_PTR)pImageBase - ntHeaders->OptionalHeader.ImageBase;
    if (delta == 0) return;

    IMAGE_BASE_RELOCATION* pReloc = (IMAGE_BASE_RELOCATION*)((BYTE*)pImageBase + relocDir.VirtualAddress);

    while (pReloc->VirtualAddress != 0) {
        DWORD numRelocations = (pReloc->SizeOfBlock - sizeof(IMAGE_BASE_RELOCATION)) / sizeof(WORD);
        WORD* pRelocData = (WORD*)(pReloc + 1);

        for (DWORD i = 0; i < numRelocations; i++) {
            WORD relocType = pRelocData[i] >> 12;
            WORD relocOffset = pRelocData[i] & 0xFFF;

            if (relocType == IMAGE_REL_BASED_HIGHLOW || relocType == IMAGE_REL_BASED_DIR64) {
                ULONG_PTR pPatch = (ULONG_PTR)((BYTE*)pImageBase + pReloc->VirtualAddress + relocOffset);
                *pPatch += delta;
            }
        }

        pReloc = (IMAGE_BASE_RELOCATION*)((BYTE*)pReloc + pReloc->SizeOfBlock);
    }
}
```

Hình 8. Function để giải quyết Relocation

1. relocDir = ...[IMAGE_DIRECTORY_ENTRY_BASERELOC];

- Lấy thông tin về Bảng Tái định vị (RVA và kích thước) từ header của file PE.

2. delta = (ULONG_PTR)pImageBase - ntHeaders->OptionalHeader.ImageBase;

- Đây là bước tính toán quan trọng nhất. Nó tính ra delta - sự chênh lệch giữa địa chỉ nạp **thực tế** (pImageBase) và địa chỉ **ưu tiên** (ImageBase). Đây chính là giá trị cần được cộng vào tất cả các địa chỉ hard-code.

3. **if (delta == 0) return;**

- Nếu delta bằng 0, nghĩa là file đã được nạp đúng địa chỉ ưu tiên, không cần làm gì cả.

4. **pReloc = ...:** Lấy con trỏ trỏ đến khối (block) tái định vị đầu tiên.

5. **while (pReloc->VirtualAddress != 0):** Vòng lặp duyệt qua từng khối trong bảng. Một khối có VirtualAddress bằng 0 báo hiệu kết thúc bảng.

6. Bên trong vòng lặp while, nó tiếp tục lặp qua từng mục tái định vị (WORD) trong khối đó:

- **relocType = pRelocData[i] >> 12;;** Tách 4 bit đầu để lấy Type.
- **relocOffset = pRelocData[i] & 0xFFF;;** Tách 12 bit sau để lấy Offset.

7. **if (relocType == IMAGE_REL_BASED_HIGHLOW || ...):** Kiểm tra xem đây có phải là loại tái định vị cần xử lý không (tức là sửa một địa chỉ đầy đủ).

8. **pPatch = (ULONG_PTR*)((BYTE*)pImageBase + pReloc->VirtualAddress + relocOffset);**

- Tính toán địa chỉ tuyệt đối của vị trí cần vá. Công thức là: Địa chỉ nạp thực tế + RVA của trang + Offset trong trang.

9. ***pPatch += delta;**

- **Đây là hành động vá lỗi:** Nó đọc giá trị địa chỉ tuyệt đối đang bị sai tại pPatch, cộng thêm delta vào, và ghi kết quả đúng trở lại.

10. **pReloc = ...:** Di chuyển con trỏ đến khối tái định vị tiếp theo.

c) *Triển khai các công việc còn lại*

Sau khi đã hoàn thành hai công cụ phức tạp nhất thì việc còn lại chỉ là phân tích và giúp cho file packed có thể tự unpack và execute trong VirtualMemory thôi.

```
bool loadAndExecute() {
    IMAGE_DOS_HEADER* dosHeader = (IMAGE_DOS_HEADER*)pMappedBase;
    IMAGE_NT_HEADERS* ntHeaders = (IMAGE_NT_HEADERS*)((BYTE*)pMappedBase + dosHeader->e_lfanew);

    // Allocate memory
    LPVOID pImageBase = VirtualAlloc(NULL, ntHeaders->OptionalHeader.SizeOfImage,
                                     MEM_COMMIT | MEM_RESERVE, PAGE_EXECUTE_READWRITE);
    if (!pImageBase) return false;

    memcpy(pImageBase, pMappedBase, ntHeaders->OptionalHeader.SizeOfHeaders);

    IMAGE_SECTION_HEADER* sections = (IMAGE_SECTION_HEADER*)((BYTE*)&ntHeaders->OptionalHeader + ntHeaders->FileHeader.
    SizeOfOptionalHeader);
    for (int i = 0; i < ntHeaders->FileHeader.NumberOfSections; i++) {
        if (sections[i].SizeOfRawData > 0) {
            memcpy((BYTE*)pImageBase + sections[i].VirtualAddress,
                  (BYTE*)pMappedBase + sections[i].PointerToRawData,
                  sections[i].SizeOfRawData);
        }
    }
    processImports(pImageBase, ntHeaders);
    processRelocations(pImageBase, ntHeaders);

    DWORD entryPoint = ntHeaders->OptionalHeader.AddressOfEntryPoint;
    void (*entry)() = (void(*)())((BYTE*)pImageBase + entryPoint);

    std::cout << "Executing PE at 0x" << std::hex << entryPoint << std::dec << "\n";
    entry();

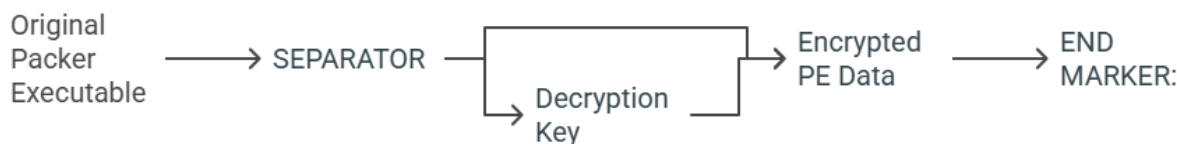
    VirtualFree(pImageBase, 0, MEM_RELEASE);
    return true;
}
```

Hình 9. Function giúp thực thi trong bộ nhớ ảo

Nói sơ qua về hàm này thì nó thực hiện những công việc sau:

1. **Cấp phát một vùng nhớ mới** có quyền thực thi (executable).
2. **Sao chép và sắp xếp** lại file PE từ **pMappedBase** vào vùng nhớ mới này theo đúng cấu trúc mà nó sẽ có khi được nạp bởi Windows.
3. **Sửa chữa** file PE trong bộ nhớ mới bằng cách xử lý **Imports** và **Relocations**.
4. **Chuyển quyền điều khiển** cho file PE đó để nó bắt đầu chạy.

Và để cho file có thể tự unpack và execute thì chúng ta cần phải “lắp ghép” các phần rời rạc này lại thành một file duy nhất. Và chúng ta sẽ tạm định nên một format cho file packed như sau:



Hình 10.Format Packed File

```
void packFile(const std::string& inputFile, const std::string& outputFile) {
    try {
        std::cout << "Packing PE file: " << inputFile << "\n";

        auto data = readFile(inputFile);
        if (data.size() < 64 || data[0] != 'M' || data[1] != 'Z') {
            throw std::runtime_error("Not a valid PE file");
        }

        generateKey();
        encryptData(data);

        // Create packed executable
        auto currentExe = readFile(getCurrentExecutablePath());
        std::vector<uint8_t> result = currentExe;

        std::string separator = "NT137.Q11.ANTT_GROUP_11";
        result.insert(result.end(), separator.begin(), separator.end());
        result.insert(result.end(), reverse_key.begin(), reverse_key.end());
        result.insert(result.end(), data.begin(), data.end());

        std::string endMarker = "PACKED_PE_MARKER";
        result.insert(result.end(), endMarker.begin(), endMarker.end());

        writeFile(outputFile + ".exe", result);
        std::cout << "Successfully created: " << outputFile << ".exe\n";

    } catch (const std::exception& e) {
        std::cerr << "Error: " << e.what() << "\n";
    }
}
```

Hình 11. Function xử lý đóng gói file

```
void unpackAndRun() {
    try {
        std::cout << "Unpacking PE using memory-mapped approach...\n";

        auto currentExe = readFile(getCurrentExecutablePath());
        std::string separator = "NT137.Q11.ANTT_GROUP_11";
        std::string magic = "PACKED_PE_MARKER";

        auto separatorPos = std::find_end(currentExe.begin(), currentExe.end(), separator.begin(), separator.end());
        size_t dataStartPos = std::distance(currentExe.begin(), separatorPos) + separator.length();
        size_t magicPos = currentExe.size() - magic.length();

        // Extract key and payload
        std::vector<uint8_t> extractedKey(currentExe.begin() + dataStartPos, currentExe.begin() + dataStartPos + 256);
        reverse_key = extractedKey;

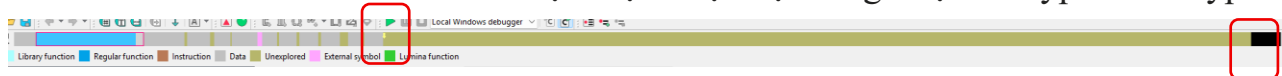
        std::vector<uint8_t> encryptedPayload(currentExe.begin() + dataStartPos + 256, currentExe.begin() + magicPos);
        decryptInMemory(encryptedPayload.data(), encryptedPayload.size());

        // Load and execute using memory mapping
        PEloaderSimple loader(encryptedPayload);
        loader.loadAndExecute();

    } catch (const std::exception& e) {
        std::cerr << "Error: " << e.what() << "\n";
    }
}
```

Hình 12. Function xử lý việc unpack và giúp file execute

Giải thích một chút về cách hoạt động của tool này. Đầu tiên, công cụ sẽ **encrypt** toàn bộ file gốc với **Substitution cipher** và chuyển toàn bộ vào phía sau của file packed. Đồng thời, cả trước và sau của đoạn được encrypt sẽ được chèn thêm **SEPARATOR** và **MARKER** để nhận diện được nội dung được encrypt để decrypt.



Hình 13. File packed được insert thêm

```

50 7B 9F 6F D7 EC EC C0 .....P{.o....
EC 1B 34 FA E2 D7 50 41 .P.).....A
4D 41 52 4B 45 52      CKED_PE_MARKER..
(Synchronized with IDA View-A)

```

Hình 14. Marker được chèn vào phần cuối file

```

31 2E 41 4E 54 54 5F 47 NT137.Q11.ANTT_G
7A 81 93 EF A6 92 C0 7D ROUP_11|z..漚...
3E 78 6B 2B 54 9F B9 3B ...d....>xk+T..;
85 F0 06 62 01 1C 18 BB 1+ \

```

Hình 15. Separator được chèn vào trước phần được encrypt

Có thể click vào đầu và cuối của đoạn màu vàng trong IDA (Hình 13) để xem nhanh.

Sau khi nhúng xong thì sẽ tạo ra một file **.EXE** duy nhất có thể thực thi được. File này đồng thời có thể tự **unpack** khi thực thi. Vậy làm thế nào mà nó có thể tự **unpack**?

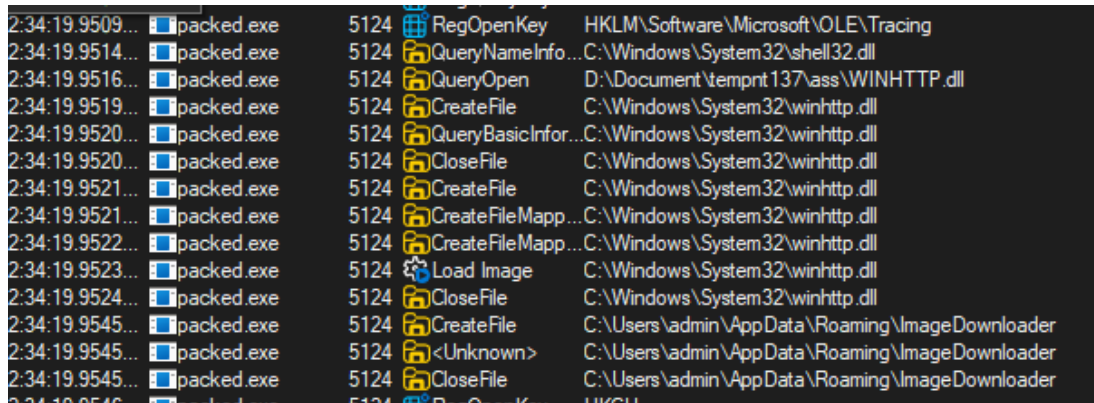
Đơn giản là trong hàm **main()** chúng ta tạo thêm một **trigger** để tự **execute** function **unpack** (Hình 12). Lúc này chương trình sẽ tự ghép lại các IAT, xử lý Relocations, Jump đến OEP để thực thi,... và như vậy là hoàn thành.

Address	Ordinal	Name	Library
libgcc_s_seh-1			
000000014000B3A0		_Unwind_Resume	libgcc_s_seh-1
KERNEL32			
000000014000B3B0		CloseHandle	KERNEL32
000000014000B3B8		CreateFileA	KERNEL32
000000014000B3C0		CreateFileMappingA	KERNEL32
000000014000B3C8		DeleteCriticalSection	KERNEL32
000000014000B3D0		EnterCriticalSection	KERNEL32
000000014000B3D8		GetLastError	KERNEL32
000000014000B3E0		GetModuleFileNameA	KERNEL32
000000014000B3E8		GetProcAddress	KERNEL32
000000014000B3F0		GetTempPathA	KERNEL32
000000014000B3F8		InitializeCriticalSection	KERNEL32
000000014000B400		LeaveCriticalSection	KERNEL32
000000014000B408		LoadLibraryA	KERNEL32
000000014000B410		MapViewOfFile	KERNEL32
000000014000B418		SetUnhandledExceptionFilter	KERNEL32
000000014000B420		Sleep	KERNEL32
000000014000B428		TlsGetValue	KERNEL32
000000014000B430		UnmapViewOfFile	KERNEL32
000000014000B438		VirtualAlloc	KERNEL32
000000014000B440		VirtualFree	KERNEL32
000000014000B448		VirtualProtect	KERNEL32
000000014000B450		VirtualQuery	KERNEL32
msvcrt			

Hình 16. Các DLL được gọi

Có thể kiểm tra với IDA, hoàn toàn không thấy các **WinHTTP** được Import mà chỉ còn là các DLL của **kernel32.dll**

Tuy nhiên thì nếu kiểm tra với ProcMon thì vẫn sẽ hiển thị các winhttp.dll chứ không hoàn toàn mất đi.



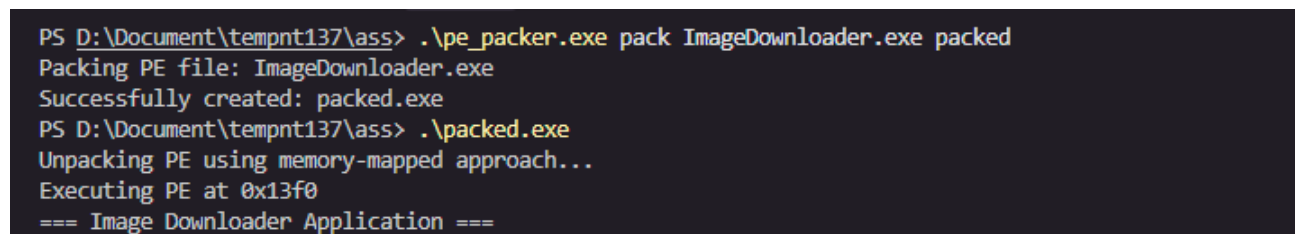
Hình 17. Các winhttp.dll được gọi

Lí do là vì Stub của packer chỉ là công cụ trung gian để gọi đến các DLL thôi nên khi phân tích động thì vẫn sẽ bị lộ.

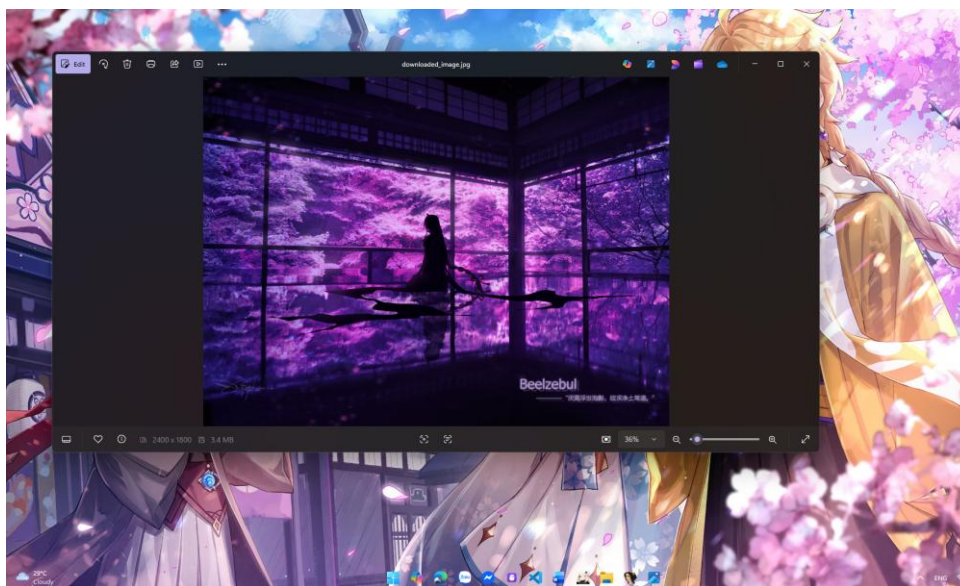
4. Test với VirusTotal

Sau khi hoàn thành công cụ, chúng ta sẽ thử pack với file ở bài tập trước (ImageDownloader.exe)

Compile: g++ -o pe_packer.exe pe_packer_simple.cpp -O2 -s



Hình 18. Usage



Hình 19. Kết quả file vẫn có thể tự unpack và execute

ware\Microsoft\Windows\CurrentVersion\Run

Name	Type	Data
(Default)	REG_SZ	(value not set)
ImageDownloader	REG_SZ	C:\Users\admin\AppData\Roaming\ImageDownloader\show_image.bat

Hình 20. Kiểm tra các Registry và vẫn đăng ký được

```
PS D:\Document\tempnt137\ass> Get-FileHash -Path .\packed.exe

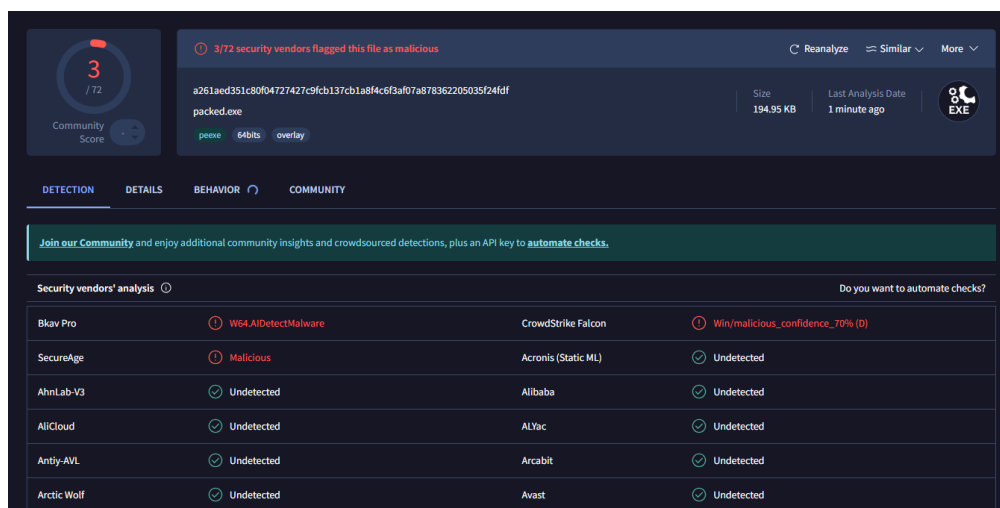
Algorithm      Hash
-----
SHA256         A261AED351C80F04727427C9FCB137CB1A8F4C6F3AF07A878362205035F24FDF
Path           D:\Document\tempnt137\ass\packed.exe

PS D:\Document\tempnt137\ass> Get-FileHash -Path .\ImageDownloader.exe

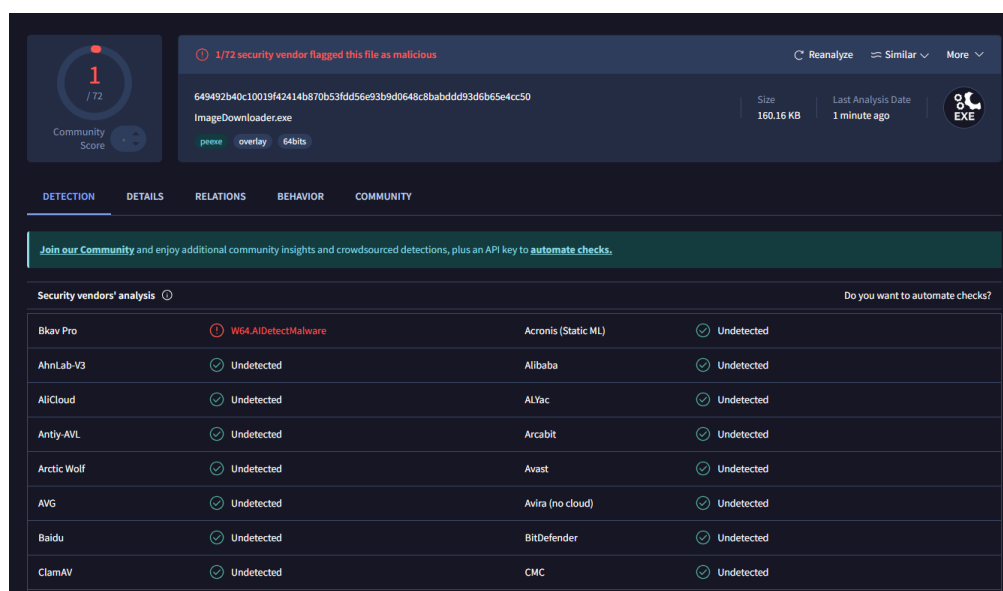
Algorithm      Hash
-----
SHA256         649492B40C10019F42414B870B53FDD56E93B9D0648C8BABDD93D6B65E4CC50
Path           D:\Document\tempnt137\ass\ImageDownloader.exe
```

Hình 21. Checksum

Và sau cùng là sẽ kiểm tra với **VirusTotal**. Kết quả thì file gốc bị phát hiện 1/72 và file packed bị shutdown với số điểm 3/72 ;-;



Hình 22. Packed File



Hình 23. Original File

5. Danh mục tham khảo

Defeating Anti-Debugging Techniques for Malware Analysis Using a Debugger. Được truy lục từ https://www.astesj.com/publications/ASTESJ_0506142.pdf

PE File Structure. Được truy lục từ <https://www.curlybrace.com/archive/PE%20File%20Structure.pdf>

Tìm hiểu PE file qua các ví dụ cơ bản | Oday in {REA_TEAM}. Được truy lục từ <https://kienmanowar.wordpress.com/category/my-tutorials/tim-hieu-pe-file-qua-cac-vi-du-co-ban/>

Writing a simple self-injecting packer | David's blog. (n.d.). Retrieved from <https://dsxm.de/simple-packer/>

HẾT